Claude Code Insights

243 messages across 41 sessions (61 total) | 2026-02-13 to 2026-02-20

At a Glance

What's working: You've developed a strong plan-first workflow where you use Claude as a design partner before jumping into implementation — this is paying off with a high success rate across your Revit add-in suite. Your iterative UI refinement sessions are particularly effective, using tight feedback loops to converge on polished WPF layouts quickly. Impressive Things You Did →

What's hindering you: On Claude's side, it repeatedly jumps into implementation without checking Revit API feasibility first (read-only parameters, active document restrictions), leading to wasted cycles and full reverts. On your side, planning sessions frequently get interrupted before completion — likely because the output scope or format doesn't match what you need — and there's a recurring WSL-vs-Windows path issue that a one-time configuration fix could eliminate. Where Things Go Wrong →

Quick wins to try: Create a custom `/build` skill with your exact Windows MSBuild command so Claude never defaults to Linux paths again. You could also add a hook that auto-runs your build after edits, catching errors immediately instead of manually triggering builds each time. Features to Try →

Ambitious workflows: As models get more capable, you'll be able to run full plan-to-implementation pipelines in a single session — Claude generates the plan, validates it against Revit API constraints, implements across all files, and build-verifies at each phase without you needing to babysit. Your multi-add-in consolidation work will also benefit from parallel subtask execution, where independent file groups get refactored simultaneously and merged with a single build pass. On the Horizon →

What You Work On | How You Use CC | Impressive Things | Where Things Go Wrong

Features to Try | New Usage Patterns | On the Horizon | Team Feedback

243	+14,991/-1,569	209	6	40.5
MESSAGES	LINES	FILES	DAYS	MSGS/DAY

What You Work On

TurboDriver - LED Driver Selection Add-in

~12 sessions

Built and iteratively refined a Revit add-in for dynamic LED driver selection, including pulling driver suggestions from project data, fixture grouping, circuit filtering, and a polished WPF UI with collapsible sections and styled layouts. Claude handled extensive multi-file edits across UI, logic, and data layers with clean builds.

TurboNumber - Panel Circuit Numbering Add-in

~8 sessions

Developed a Revit add-in for panel circuit numbering with manual/auto toggle modes, custom room sorting, and parameter writing. Work included planning phases, iterating on Revit API limitations (read-only circuit numbers), and building out UI features like sidebars and column configurations.

TurboZones - Zone Label Resolution Add-in

~3 sessions

Implemented and refined a TurboZones Revit command for room/zone label resolution, including fixing room name retrieval, parenthesis trimming, and comment handling logic. Claude successfully built the feature and updated documentation.

TurboCompact & TurboBubble - Utility Add-ins

~5 sessions

Built TurboCompact for document management (working around Revit's active document close restriction), planned and implemented parameter-based tag selection for TurboBubble, and fixed wall sconce placement and RPS wire routing bugs. Some sessions involved debugging Revit API constraints.

TurboSuite Consolidation & Project Infrastructure

~5 sessions

Consolidated sub-directories into a unified TurboSuite structure, resolved namespace conflicts and build configuration issues, aligned project structures across sibling add-ins, and generated CLAUDE.md documentation. Claude produced detailed multi-phase consolidation plans and handled cross-project refactoring.

WHAT YOU WANTED

Planning	14
Bug Fix	12
Feature Impleme...	11
Ui Refinement	9
Feature Modifica...	7
Business Logic C...	4

TOP TOOLS USED

Read	415
Edit	376
Bash	172
Write	131
TaskUpdate	108
TaskCreate	55

LANGUAGES

Markdown	80

SESSION TYPES

Iterative Refinem...	22
Single Task	13
Exploration	2
Quick Question	2
Multi Task	2

How You Use Claude Code

You are a **plan-first, iterate-fast developer** who uses Claude Code as both an architect and implementer for a suite of Revit add-ins. A striking pattern is how frequently you kick off sessions by asking Claude to review a spec and generate a development plan — 14 of your sessions are explicitly planning-focused, and you often **interrupt plan generation mid-stream** to redirect or refine the approach before implementation begins (at least 5-6 sessions show this pattern). This suggests you use Claude's planning output as a thinking tool, not necessarily expecting to consume the full output before course-correcting.

Once you move into implementation, you adopt a **rapid iterative refinement style**, often cycling through 8-15+ files in a single session with successive rounds of UI tweaks, logic changes, and

build verification. Sessions like your TurboDriver work show you pushing through driver selection logic, voltage filtering, fixture limits, column layouts, and styling — all in continuous back-and-forth. You're comfortable letting Claude make multi-file changes autonomously (Read: 415, Edit: 376, Write: 131 tool calls reflect heavy autonomous editing), but you **stay closely engaged and correct mistakes quickly** — whether it's sorting logic, color choices being too dark, or naming mismatches. You rarely let incorrect approaches persist for long.

A notable friction pattern is that **Claude repeatedly struggles with environment-specific issues** in your workflow — using Linux/WSL paths instead of Windows paths for builds, and sometimes misjudging Revit API constraints (like the read-only circuit number parameter that should have been flagged as a risk upfront). Despite 16 instances of wrong-approach friction and a 9% dissatisfaction rate, your overall outcomes are strong (83% fully or mostly achieved), indicating you're skilled at steering Claude back on track. You also leverage the **sub-task system heavily** (TaskCreate: 55, TaskUpdate: 108), suggesting you break complex work into managed chunks rather than issuing monolithic requests.

Key pattern: You lead with spec-driven planning sessions that you frequently interrupt to course-correct, then shift into rapid multi-file implementation cycles where you stay closely engaged to catch and redirect mistakes.

USER RESPONSE TIME DISTRIBUTION

2-10s	10
10-30s	18
30s-1m	12
1-2m	15
2-5m	34
5-15m	45
>15m	6

Median: 211.7s • Average: 279.8s

MULTI-CLAUDING (PARALLEL SESSIONS)

10	16	16%
OVERLAP EVENTS	SESSIONS INVOLVED	OF MESSAGES

You run multiple Claude Code sessions simultaneously. Multi-clauding is detected when sessions overlap in time, suggesting parallel workflows.

USER MESSAGES BY TIME OF DAY	PT (UTC-8)
Morning (6-12)	34
Afternoon (12-18)	192
Evening (18-24)	17
Night (0-6)	0

TOOL ERRORS ENCOUNTERED	
User Rejected	31
Command Failed	26
Other	3
File Not Found	1

Impressive Things You Did

Over the past week, you've been intensively developing a suite of Revit add-ins (TurboSuite), using Claude Code heavily for planning, implementation, and iterative refinement across 41 sessions.

Plan-First Development Approach

You consistently start with dedicated planning sessions before implementation, having Claude analyze specs and produce detailed multi-phase plans. With 14 planning-focused sessions, you're using Claude as a design partner to think through complex Revit API challenges before writing code, which contributes to your high 83% full/mostly achieved rate.

Rapid Iterative UI Refinement

You excel at using Claude for tight feedback loops on WPF styling, column layouts, and visual hierarchy — often refining details like header colors, section ordering, and button sizing across multiple rounds in a single session. This iterative approach lets you treat Claude like a live design tool, quickly converging on polished results.

Multi-Tool Complex Codebase Navigation

You're leveraging Claude's full toolkit effectively, with heavy Read (415) and Edit (376) usage combined with task orchestration (208 task-related calls) to manage changes spanning 8-15+ files per session. Your TurboDriver and TurboNumber sessions show you confidently directing Claude through intricate cross-file refactors with clean builds each time.

WHAT HELPED MOST (CLAUDE'S CAPABILITIES)	
Good Explanations	15
Multi-file Changes	15
Correct Code Edits	6
Good Debugging	4

OUTCOMES	
Not Achieved	1
Partially Achieved	6
Mostly Achieved	12
Fully Achieved	22

Where Things Go Wrong

Your sessions show a recurring pattern of wrong initial approaches, platform misconfigurations, and excessive planning iterations that slow down your workflow.

Repeated Wrong Platform/Path Configuration

Claude keeps using WSL/Linux-style paths for Windows builds, causing failed build attempts that you have to correct. Consider adding explicit platform instructions to your CLAUDE.md or project configuration so Claude defaults to Windows paths.

- Multiple sessions had initial builds fail because Claude used WSL path format instead of Windows-style paths for dotnet.exe/MSBuild, requiring retries each time
- Solution file had x64 platform configurations mismatched with the project's AnyCPU configurations, causing additional build troubleshooting

Costly Wrong Approaches Before Feasibility Checks

Claude often dives into implementation before validating whether an approach is feasible with the Revit API, leading to multiple failed attempts and even full reverts. You could prompt Claude to research API constraints and flag risks before writing any code.

- You spent an entire session on circuit number zero-padding only to discover Revit's circuit number parameter is read-only — Claude should have flagged this risk upfront instead of attempting two failed approaches
- The Circuit Naming parameter took three iterations (SelectedIndex, SetValueString, probing ElementIds) before landing on hardcoded built-in ElementIds, wasting significant back-and-forth

Interrupted and Redundant Planning Sessions

You have at least 5-6 sessions where Claude was generating a plan but you interrupted before completion, suggesting the planning output format or scope isn't matching your needs. Try giving Claude a plan template or length constraint upfront to get usable output faster.

- Planning sessions for TurboNumber, TurboCompact, and TurboZones were all interrupted before completion, resulting in partially_achieved outcomes that required follow-up sessions
- You had multiple empty sessions (clear/exit with no requests) interspersed with planning sessions, suggesting friction in getting Claude to produce plans in the right format on the first try

PRIMARY FRICTION TYPES

Wrong Approach	16
Buggy Code	5
Misunderstood R...	4
Api Errors	3
User Rejected Ac...	1

INFERRED SATISFACTION (MODEL-ESTIMATED)

Dissatisfied	9
Likely Satisfied	77
Satisfied	3

Existing CC Features to Try

Suggested CLAUDE.md Additions

Just copy this into Claude Code to add it to your CLAUDE.md.

Copy All Checked

☑

```
Always use Windows-style paths (e.g., `C:\Program Files\dotnet\dotnet.exe`) for build commands. Never use WSL/Linux-style paths (e.g., `/mnt/c/...`) when running MSBuild or dotnet commands.
```

Copy

WSL-style paths caused build failures in at least 4 separate sessions, requiring manual correction each time.

☑

```
When working with Revit API parameters, always check whether a parameter is read-only or computed before attempting to write to it. For ElementId storage type parameters, probe valid ElementId values rather than assuming string or integer assignment will work.
```

Copy

Multiple sessions had friction from attempting writes to read-only Revit parameters or using wrong value types for ElementId parameters, wasting significant iteration time.

- [x]

```
When the user asks to explain something, provide an explanation first. Do not assume they want a code change, behavior modification, or memory update unless explicitly asked.
```

Copy

Claude repeatedly misunderstood explanation requests as change requests, causing frustration.

- [x]

```
The active Revit document cannot be closed via `doc.Close()` from the API. If document closing is needed, use the Idling event pattern or defer to user action.
```

Copy

Two sessions hit this exact Revit API limitation, with multiple failed approaches before discovering the constraint.

- [x]

```
This is a Windows development environment targeting Revit add-ins. Build platform is typically AnyCPU. Always verify solution/project platform configurations match before building.
```

Copy

Platform mismatch (x64 vs AnyCPU) and environment assumptions caused build failures across sessions.

Just copy this into Claude Code and it'll set it up for you.

Custom Skills

Reusable prompts that run with a single /command

Why for you: You have 14 planning sessions that follow the same pattern (read spec, explore codebase, write plan). A /plan skill would standardize this and prevent interruptions from incomplete outputs.

Copy

```
mkdir -p .claude/skills/plan && cat > .claude/skills/plan/SKILL.md << 'EOF'
# Plan Generation Skill
1. Read the spec file provided by the user
2. Explore the relevant codebase using Read and Grep
3. Ask up to 3 clarifying questions before writing
4. Output the plan in numbered phases with concrete file paths
5. Keep total plan under 200 lines
EOF
```

Custom Skills

A /build skill to standardize the Windows build command

Why for you: Build path errors recurred in 4+ sessions. A /build skill would encode the correct Windows dotnet path once and eliminate this friction permanently.

Copy

```
mkdir -p .claude/skills/build && cat > .claude/skills/build/SKILL.md << 'EOF'
# Build Skill
Run the build using Windows-style paths only:
```
dotnet build "<solution-path>" --configuration Debug
```
Never use WSL paths like /mnt/c/. Always use C:\ style paths.
If build fails, check platform config (AnyCPU vs x64) mismatch first.
EOF
```

Hooks

Auto-run shell commands at lifecycle events like after edits

Why for you: You have 0 commits across 41 sessions and heavy Edit/Write usage. A post-edit hook could auto-run builds so you catch errors immediately instead of discovering them later.

Copy

```
// Add to .claude/settings.json:
{
  "hooks": {
    "postToolUse": [
      {
        "matcher": "Edit|Write",
        "command": "echo 'Remember: run build to verify changes'"
      }
    ]
  }
}
```

New Ways to Use Claude Code

Just copy this into Claude Code and it'll walk you through it.

Plan sessions keep getting interrupted

Structure planning sessions with explicit output length limits and checkpoints.

At least 5-6 planning sessions ended with user interruptions before the plan was complete. This suggests Claude's plans are either too long or not structured in digestible chunks. Try asking for plans in phases, or set explicit constraints like 'keep the plan under 100 lines' so you get a complete deliverable every time.

PASTE INTO CLAUDE CODE:

```
Read the spec at [path] and create a development plan. Keep it under 100 lines total.
Use numbered phases. Stop after writing the plan — don't start implementing.
```

Copy

Front-load API feasibility checks

Before implementing Revit API features, ask Claude to verify parameter writability and API constraints first.

Multiple sessions burned significant time attempting impossible operations (read-only circuit numbers, active doc closing). A quick feasibility check prompt before implementation would save hours. This pattern applies to any Revit API work where you're writing to parameters or manipulating documents.

PASTE INTO CLAUDE CODE:

```
Before implementing anything, check: Is [parameter name] writable via the Revit API?
Are there any known restrictions on this operation? List any limitations or
alternative approaches. Only proceed to implementation after confirming feasibility.
```

Copy

Batch your UI refinement iterations

Combine multiple small UI tweaks into a single structured request instead of one-at-a-time.

9 sessions focused on UI refinement with iterative back-and-forth (colors too dark, columns wrong width, button sizing). Grouping these into a single request with specific values (hex codes, pixel widths) upfront would reduce round trips. Your most successful sessions were the ones where you provided all specs at once.

PASTE INTO CLAUDE CODE:

```
Make these UI changes to [file]: 1) Header background color: #E8E8E8 2) Column
widths: Name=200, Type=150, Load=100 3) Button padding: 8px horizontal, 4px vertical.
Build after all changes and report any errors.
```

Copy

On the Horizon

Your Claude Code usage reveals a mature AI-assisted development workflow heavy on planning and iterative refinement—now ready to level up with autonomous, parallelized agent patterns.

Autonomous Plan-to-Implementation Pipeline

With 14 planning sessions often interrupted before completion, you can have Claude autonomously generate a plan, validate it against your codebase constraints, and then execute the implementation in one continuous flow. Instead of separate plan-then-implement sessions, a single prompt can drive the full cycle with built-in checkpoints and MSBuild verification after each phase.

Getting started: Use Claude Code's Task tool for parallel subtasks—one for planning, one for implementation, one for build validation—so they hand off automatically.

PASTE INTO CLAUDE CODE:

```
Review the spec at [spec_file_path] and the existing codebase structure. First, generate a phased implementation plan. Then immediately begin implementing phase by phase. After each phase: 1) Run the Windows build command to verify compilation, 2) If build fails, fix before proceeding, 3) Log what was completed. Use subtasks to parallelize independent phases. Do NOT stop between planning and implementation—treat this as one continuous workflow. If any Revit API limitation is detected (read-only parameters, active document restrictions), flag it BEFORE writing code and propose alternatives.
```

Copy

Revit API Constraint Pre-Validation Agent

Your biggest friction source is discovering Revit API limitations mid-implementation—read-only parameters, active document restrictions, ElementId storage types. An upfront validation pass could catch these blockers before any code is written, saving the multiple iteration cycles you've been experiencing. Claude can systematically audit proposed changes against known Revit API constraints.

Getting started: Create a CLAUDE.md section documenting known Revit API pitfalls, then prompt Claude to run a constraint check before any implementation session.

PASTE INTO CLAUDE CODE:

```
Before implementing anything, perform a Revit API feasibility audit for this feature: [describe feature]. Check the codebase and CLAUDE.md for known constraints. Specifically validate: 1) Are all target parameters writable (not computed/read-only)? 2) Does this require active document operations that Revit restricts? 3) Are we using correct parameter storage types (ElementId vs string vs int)? 4) Will this work for both face-based and point-based fixture families? Output a risk assessment with RED/YELLOW/GREEN for each concern
```

```
before writing any code. If any RED items exist, propose alternative approaches first.
```

Copy

Parallel Multi-File Refactoring with Test Builds

Your strongest sessions involve sweeping changes across 8-15+ files with iterative build verification. Instead of sequential edit-build-fix cycles, Claude can spawn parallel subtasks for independent file groups, merge results, and run a single build validation pass. This could cut your multi-file refactoring time significantly while catching cross-file issues like namespace conflicts and stale references upfront.

Getting started: Leverage TaskCreate to split independent file changes into parallel subtasks, with a final subtask that runs MSBuild and fixes any integration issues.

PASTE INTO CLAUDE CODE:

```
Implement the following changes across the TurboSuite codebase: [describe changes].
Approach: 1) First, analyze all affected files and group them by independence (files that
can be changed without affecting each other). 2) Use parallel subtasks for each independent
group. 3) After all subtasks complete, run the Windows build: dotnet build [solution_path].
4) If build fails, analyze ALL errors holistically—check for namespace conflicts, missing
references, platform mismatches (AnyCPU vs x64), and stale variable references. Fix all
errors in one pass rather than one at a time. 5) IMPORTANT: Use Windows-style paths
(backslashes), not WSL/Linux paths. Confirm successful build before reporting completion.
```

Copy

"Developer spent hours trying to make Revit zero-pad circuit numbers, only to discover the parameter is completely read-only — and had to revert every line of code"

In a painful early session, Claude tried multiple approaches to implement circuit number zero-padding in a Revit add-in. After iterating through direct parameter writes and naming-switch fallbacks, they hit a wall: Revit's circuit number parameter is fully computed and read-only. The entire session's work was reverted, achieving nothing — a classic 'fighting the API' moment.